By using a separate `DbContext` for each purpose, you can avoid bugs in your tests due to EF Core caching data without writing it to the database. With this approach, you can be sure that any data read in the second `DbContext` was persisted to the underlying in-memory database provider.

This was a brief introduction to using the SQLite provider as an in-memory database provider and EF Core testing in general, but if you follow the setup shown in listing 36.13, it should take you a long way. The source code for this chapter shows how you can combine this code with a custom `WebApplicationFactory` to use an in-memory database for your integration tests. For more details on testing EF Core, including additional options and strategies, see *Entity Framework Core in Action*, 2nd ed., by Jon P. Smith (Manning, 2021).

Summary

- Use the `DefaultHttpContext` class to unit-test your custom middleware components. If you need access to the response body, you must replace the default `Stream.Null` with a `MemoryStream` instance and read the stream manually after invoking the middleware.
- API controllers, minimal APIs, and Razor Page models can be unit-tested like other classes, but they should generally contain little business logic, so it may not be worth the effort. For example, the API controller is tested independently of routing, model validation, and filters, so you can’t easily test logic that depends on any of these aspects.